

C# - Phần 10.7: LINQ Skip(), Take() và Pagination

Lý thuyết - Khái niệm phân trang - Câu hỏi - Bài tập - Đáp án - Lưu ý

Mục tiêu: Hiểu Skip() và Take(), hiểu pageNumber/pageSize/totalItems/totalPages, công thức phân trang, thứ tự OrderBy -> Skip -> Take, kiểu dữ liệu qua chuỗi LINQ và các tình huống biên.

PHẦN A - LÝ THUYẾT SKIP() VÀ TAKE()

1. Skip(n)

Skip(n) bỏ qua n phần tử đầu tiên của sequence và trả về các phần tử còn lại.

```
var result =  
    products.Skip(3);
```

Nếu có P01...P10 thì Skip(3) bỏ P01, P02, P03 và kết quả bắt đầu từ P04.

2. Take(n)

Take(n) lấy tối đa n phần tử đầu tiên của sequence hiện tại.

```
var result =  
    products.Take(4);
```

Nếu sequence có ít hơn n phần tử, Take(n) lấy hết những gì đang có và không throw exception.

3. Skip() + Take()

```
var result =  
    products  
        .Skip(2)  
        .Take(3);
```

Thứ tự là: bỏ 2 phần tử đầu, sau đó lấy 3 phần tử tiếp theo.

4. Kiểu kết quả

```
products.Skip(2)  
→ IEnumerable<Product>  
  
products.Take(3)  
→ IEnumerable<Product>  
  
products.Skip(2).Take(3)  
→ IEnumerable<Product>
```

Muốn List<Product> thì thêm ToList().

5. Không sửa collection gốc

Skip() và Take() không xóa hay cắt danh sách products gốc; chúng tạo sequence kết quả mới.

Ghi nhớ: Skip != Remove và Take != thay đổi List gốc.

6. Trường hợp biên

```
Skip(100)  
→ sequence rỗng nếu collection chỉ có ít phần tử'  
  
Take(100)  
→ lấy hết phần tử đang có  
  
Take(0)  
→ sequence rỗng
```

PHẦN B - KHÁI NIỆM PAGINATION / PHÂN TRANG

7. Trang là gì?

Pagination là chia một tập dữ liệu lớn thành nhiều trang nhỏ để chỉ lấy một phần dữ liệu mỗi lần.

Ví dụ: 25 Product, pageSize = 5

Trang 1 → Product 1 - 5
Trang 2 → Product 6 - 10
Trang 3 → Product 11 - 15
Trang 4 → Product 16 - 20
Trang 5 → Product 21 - 25

8. pageNumber và pageSize

pageNumber
→ trang hiện tại muốn lấy

pageSize
→ số phần tử tối đa trên mỗi trang

Ví dụ pageNumber = 2 và pageSize = 5 nghĩa là lấy trang 2, mỗi trang có 5 phần tử.

9. Công thức phân trang

```
.Skip((pageNumber - 1) * pageSize)  
.Take(pageSize)
```

Phần Skip tính số phần tử thuộc các trang trước cần bỏ qua. Take(pageSize) mới lấy dữ liệu của trang hiện tại.

10. Vì sao pageNumber - 1?

pageNumber - 1 là số trang nằm trước trang hiện tại.

Trang 1:
(1 - 1) * pageSize = 0
→ không bỏ gì

Trang 2:
(2 - 1) * pageSize
→ bỏ 1 trang trước

Trang 3:
(3 - 1) * pageSize
→ bỏ 2 trang trước

11. Ví dụ trang 2, pageSize = 3

```
int pageNumber = 2;  
int pageSize = 3;  
  
var page =  
    products  
        .Skip((pageNumber - 1) * pageSize)  
        .Take(pageSize);
```

Tính ra Skip(3).Take(3), nên nếu dữ liệu là P01...P10 thì kết quả là P04, P05, P06.

12. Các khái niệm liên quan

pageNumber → trang hiện tại
pageSize → số phần tử mỗi trang
totalItems → tổng số phần tử
totalPages → tổng số trang
offset → số phần tử bỏ qua
limit → số phần tử lấy

Trong LINQ, offset gắn với Skip và limit gắn với Take.

13. totalPages

```
int totalPages =  
    (int)Math.Ceiling(  
        (double)totalItems / pageSize);
```

Ví dụ 23 phần tử, pageSize = 5 thì có 5 trang: 4 trang đầy và 1 trang còn 3 phần tử.

14. Vì sao OrderBy trước Skip/Take?

Phân trang nên dựa trên một thứ tự xác định. Vì vậy trong backend thường sắp xếp trước, rồi mới bỏ và lấy dữ liệu.

```
products  
    .OrderBy(product => product.Id)  
    .Skip((pageNumber - 1) * pageSize)  
    .Take(pageSize);
```

OrderBy

→ xác định thứ tự

Skip

→ bỏ dữ liệu các trang trước

Take

→ lấy dữ liệu trang hiện tại

15. Thứ tự LINQ có ý nghĩa

Sắp toàn bộ rồi phân trang khác với phân trang trước rồi mới sắp phần dữ liệu đã lấy.

Đúng tư duy pagination:

OrderBy → Skip → Take

16. Validation

```
pageNumber >= 1  
pageSize > 0
```

Nếu pageNumber = 0, công thức offset thành số âm. Với LINQ Skip, count không dương không phải là cách nghiệp vụ hợp lệ; hệ thống nên chặn pageNumber < 1.

```
if (pageNumber < 1)  
{  
    throw new ArgumentOutOfRangeException(  
        nameof(pageNumber),  
        "Page number must be at least 1.");  
}
```

PHẦN C - THEO DÕI KIỂU DỮ LIỆU

```
products  
    .Where(...)  
    .OrderBy(...)  
    .Skip(...)  
    .Take(...)  
    .Select(...)  
    .ToList();  
  
List<Product>  
    ↓ Where  
IEnumerable<Product>  
    ↓ OrderBy  
IOrderedEnumerable<Product>  
    ↓ Skip  
IEnumerable<Product>  
    ↓ Take  
IEnumerable<Product>  
    ↓ Select  
IEnumerable<TResult>  
    ↓ ToList  
List<TResult>
```

Phản xạ nên có: Khi đọc LINQ, luôn tự hỏi sau mỗi method: dữ liệu hiện tại là kiểu gì và còn là một phần tử hay nhiều phần tử?

PHẦN D - CÂU HỎI LÝ THUYẾT

1. Skip(n) dùng để làm gì?
2. Take(n) dùng để làm gì?
3. Skip(2).Take(3) hoạt động theo thứ tự nào?
4. Nếu collection có 5 phần tử nhưng Skip(10) thì chuyện gì xảy ra?
5. Nếu collection có 5 phần tử nhưng Take(10) thì chuyện gì xảy ra?
6. Skip((pageNumber - 1) * pageSize) có ý nghĩa gì?
7. Vì sao phải dùng pageNumber - 1?
8. Vì sao pagination thường nên OrderBy() trước Skip() và Take()?
9. Hãy mô tả chức năng và kiểu dữ liệu của Where -> OrderBy -> Skip -> Take -> Select -> ToList.
10. Nếu pageNumber = 0 thì có vấn đề gì về mặt nghiệp vụ?

PHẦN E - BÀI TẬP 10.7

```
List<Product> products =
    new List<Product>
    {
        new Product("P01", "Mouse", 300),
        new Product("P02", "Keyboard", 700),
        new Product("P03", "Monitor", 2500),
        new Product("P04", "Laptop", 15000),
        new Product("P05", "USB", 150),
        new Product("P06", "Cable", 300),
        new Product("P07", "Adapter", 700),
        new Product("P08", "Webcam", 1200),
        new Product("P09", "Speaker", 900),
        new Product("P10", "Headphone", 1100)
    };
};
```

- Dùng Skip(3) và hiển thị kết quả.
- Dùng Take(4) và hiển thị kết quả.
- Dùng Skip(2).Take(3).
- Sắp Price tăng dần rồi lấy 3 Product đầu tiên.
- Sắp Price giảm dần, bỏ 2 Product đầu và lấy 3 Product tiếp theo.
- Lọc Price >= 700, sắp Price tăng dần, sau đó Skip(1).Take(3).
- Với pageNumber = 2, pageSize = 3, dùng công thức pagination để lấy trang 2.
- Đổi pageNumber = 3, giữ pageSize = 3 và kiểm tra kết quả.
- Sau pagination, dùng Select() chỉ lấy Name thành List<string>.
- Test Skip(100), Take(100), Take(0).
- Kiểm tra products gốc vẫn giữ nguyên.

PHẦN F - ĐÁP ÁN CODE

```
public class Program
{
    public static void Main(string[] args)
    {
        List<Product> products =
            new List<Product>
            {
                new Product("P01", "Mouse", 300),
                new Product("P02", "Keyboard", 700),
                new Product("P03", "Monitor", 2500),
                new Product("P04", "Laptop", 15000),
                new Product("P05", "USB", 150),
            }
    }
}
```

```

        new Product("P06", "Cable", 300),
        new Product("P07", "Adapter", 700),
        new Product("P08", "Webcam", 1200),
        new Product("P09", "Speaker", 900),
        new Product("P10", "Headphone", 1100)
    };

    var skip3 = products.Skip(3);
    var take4 = products.Take(4);
    var skip2Take3 =
        products.Skip(2).Take(3);

    var cheapest3 =
        products
            .OrderBy(p => p.Price)
            .Take(3);

    var afterTop2 =
        products
            .OrderByDescending(p => p.Price)
            .Skip(2)
            .Take(3);

    var filteredPage =
        products
            .Where(p => p.Price >= 700)
            .OrderBy(p => p.Price)
            .Skip(1)
            .Take(3);

    int pageNumber = 2;
    int pageSize = 3;

    var page2 =
        products
            .OrderBy(p => p.Id)
            .Skip((pageNumber - 1) * pageSize)
            .Take(pageSize);

    pageNumber = 3;

    var page3 =
        products
            .OrderBy(p => p.Id)
            .Skip((pageNumber - 1) * pageSize)
            .Take(pageSize);

    List<string> page2Names =
        page2
            .Select(p => p.Name)
            .ToList();

    var skip100 = products.Skip(100);
    var take100 = products.Take(100);
    var take0 = products.Take(0);
}
}

```

PHẦN G - ĐÁP ÁN LÝ THUYẾT

1. Skip(n)

Bỏ qua n phần tử đầu tiên và trả về các phần tử còn lại.

2. Take(n)

Lấy tối đa n phần tử đầu tiên của sequence hiện tại.

3. Skip(2).Take(3)

Bỏ 2 phần tử trước, sau đó lấy 3 phần tử tiếp theo.

4. Skip(10) khi chỉ có 5 phần tử

Không throw; kết quả là sequence rỗng.

5. Take(10) khi chỉ có 5 phần tử

Lấy cả 5 phần tử hiện có.

6. Ý nghĩa công thức Skip

Tính số phần tử thuộc các trang trước cần bỏ qua. Take(pageSize) mới lấy trang hiện tại.

7. Vì sao pageNumber - 1

Vì đây là số trang nằm trước trang hiện tại; trang 1 có 0 trang trước.

8. Vì sao OrderBy trước pagination

Để xác định thứ tự ổn định trước khi chia trang.

9. Kiểu qua chuỗi LINQ

List<Product> -> IEnumerable<Product> -> IOrderedEnumerable<Product> -> IEnumerable<Product> -> IEnumerable<Product> -> IEnumerable<TResult> -> List<TResult>.

10. pageNumber = 0

Không hợp lệ về nghiệp vụ; cần validation pageNumber >= 1.

PHẦN H - ĐÁNH GIÁ BÀI LÀM VÀ LƯU Ý

- Bài làm đạt khoảng 8.8/10 và đã hiểu được Skip/Take cùng công thức pagination.
- Skip(3), Take(4), Skip(2).Take(3): đúng.
- OrderBy Price tăng dần rồi Take(3): đúng.
- Bài sắp Price giảm dần ban đầu thiếu Skip(2).
- Bài Price >= 700 ban đầu thiếu OrderBy() và dùng Take(2) thay vì Take(3).
- Pagination trang 2 và trang 3 dùng đúng công thức.
- Pagination + Select(Name) + ToList(): đúng.
- Skip(100), Take(100), Take(0): hiểu đúng hành vi.
- products gốc không bị thay đổi.
- Không cần using System.Net.Http.Headers trong bài này.

1. Hai đoạn cần sửa

```
var product6 =
    products
        .OrderByDescending(p => p.Price)
        .Skip(2)
        .Take(3);

var product7 =
    products
        .Where(p => p.Price >= 700)
        .OrderBy(p => p.Price)
        .Skip(1)
        .Take(3);
```

2. Ba điểm cần nhớ

1. Pagination:
OrderBy(...)
.Skip((pageNumber - 1) * pageSize)
.Take(pageSize)
2. pageNumber - 1
= số trang nằm trước trang hiện tại
3. Skip/Take
không thay đổi collection gốc

Kết luận: Sau khi sửa hai truy vấn trên, Phần 10.7 - Skip(), Take() và Pagination được xem là ĐẠT. Bài tiếp theo là 10.8 - Distinct().